Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ЛАБОРАТОРНАЯ РАБОТА № 2

«Объектно-ориентированные возможности языка Python»

ДИСЦИПЛИНА: «Парадигмы и конструкции конструкции языков программирования»

Выполнил: студент гр. ИУ5-33Б _____ (Ахметзянова Г.Т.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Оргсян О.Г.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Цель: изучение объектно-ориентированных возможностей языка Python.

Задание: В виртуальном окружении с установленным сторонним pip-пакетом создайте консольное приложение с модульной структурой: пакет lab_python_oop и точку входа main.py.

Реализуйте в отдельных файлах иерархию классов (абстрактную фигуру, класс цвета, прямоугольник, круг и наследуемый от прямоугольника квадрат) с вычислением площади и методом __repr__.

В файле main.py инициализируйте фигуры с размерами по номеру вашего варианта, выведите информацию о них в консоль, задействуйте метод установленного пакета и опционально напишите unit-тесты.

Выполнение работы.

1. Написание программы

```
1
2   from lab_python_oop.figure import Figure
3   from lab_python_oop.figure_color import FigureColor
4   from lab_python_oop.rectangle import Rectangle
5   from lab_python_oop.circle import Circle
6   from lab_python_oop.square import Square
7
```

Рисунок 1 – Содержание файла __init__.py

```
1
2   import math
3
4   from lab_python_oop.figure import Figure
5   from lab_python_oop.figure_color import FigureColor
6
7   class Circle(Figure):
8       figure_name = "Круг"
9
10      def __init__(self, radius, color):
11          self.radius = radius
12          self.figure_color = FigureColor(color)
13
14      def area(self):
15          return math.pi * self.radius ** 2
16
17      @classmethod
18      def get_figure_name(cls):
19          return cls.figure_name
20
21      def __repr__(self):
22          return (
23              "Фигура: {}. Радиус: {}. "
24              "Цвет: {}. Площадь: {:.2f}"
25          ).format(
26              self.get_figure_name(),
27              self.radius,
28              self.figure_color.color,
29              self.area()
30          )
```

Рисунок 2 – Содержание файла circle.py

```python

from abc import ABC, abstractmethod


class Figure(ABC):
    @abstractmethod
    def area(self):
        pass
```

Рисунок 3 – Содержание файла figure.py

```python

class FigureColor:
    def __init__(self, color):
        self._color = color

    @property
    def color(self):
        return self._color

    @color.setter
    def color(self, value):
        self._color = value
```

Рисунок 4 – Содержание файла figure_color.py

```python

from lab_python_oop.figure import Figure
from lab_python_oop.figure_color import FigureColor


class Rectangle(Figure):
    figure_name = "Прямоугольник"

    def __init__(self, width, height, color):
        self.width = width
        self.height = height
        self.figure_color = FigureColor(color)

    def area(self):
        return self.width * self.height

    @classmethod
    def get_figure_name(cls):
        return cls.figure_name

    def __repr__(self):
        return (
            "Фигура: {}. Ширина: {}. Высота: {}. "
            "Цвет: {}. Площадь: {:.2f}"
        ).format(
            self.get_figure_name(),
            self.width,
            self.height,
            self.figure_color.color,
            self.area()
```

Рисунок 5 – Содержание файла rectange.py

```python

from lab_python_oop.rectangle import Rectangle


class Square(Rectangle):
    figure_name = "Квадрат"

    def __init__(self, side, color):
        super().__init__(side, side, color)
```

Рисунок 6 – Содержание файла square.py

```python
from lab_python_oop.rectangle import Rectangle
from lab_python_oop.circle import Circle
from lab_python_oop.square import Square

from colorama import Fore, Style, init

N = 1


def main():
    init(autoreset=True, convert=True)

    rectangle = Rectangle(N, N, "синий")
    circle = Circle(N, "зелёный")
    square = Square(N, "красный")

    print(Fore.BLUE + str(rectangle))
    print(Fore.GREEN + str(circle))
    print(Fore.RED + str(square))

    print(Style.BRIGHT + "Лабораторная работа: ООП в Python")


if __name__ == "__main__":
    main()
```

Рисунок 7 – Содержание файла main.py

```python

import math
import unittest

from lab_python_oop.figure import Figure
from lab_python_oop.figure_color import FigureColor
from lab_python_oop.rectangle import Rectangle
from lab_python_oop.circle import Circle
from lab_python_oop.square import Square


class TestFigures(unittest.TestCase):

    def test_rectangle_area(self):
        rectangle = Rectangle(4, 5, "синий")
        self.assertEqual(rectangle.area(), 20)

    def test_circle_area(self):
        circle = Circle(2, "зелёный")
        self.assertAlmostEqual(circle.area(), math.pi * 4)

    def test_square_area(self):
        square = Square(4, "красный")
        self.assertEqual(square.area(), 16)

    def test_colors(self):
        rectangle = Rectangle(2, 3, "синий")
        self.assertEqual(rectangle.figure_color.color, "синий")

    def test_color_property(self):
        color = FigureColor("синий")
        color.color = "красный"
        self.assertEqual(color.color, "красный")

    def test_figure_names(self):
        self.assertEqual(Rectangle.get_figure_name(), "Прямоугольник")
        self.assertEqual(Circle.get_figure_name(), "Круг")
        self.assertEqual(Square.get_figure_name(), "Квадрат")
```

```python

    def test_repr(self):
        square = Square(3, "красный")
        self.assertIn("Квадрат", repr(square))
        self.assertIn("красный", repr(square))
        self.assertIn("9.00", repr(square))

    def test_figure_is_abstract(self):
        with self.assertRaises(TypeError):
            Figure()


if __name__ == "__main__":
    unittest.main()
```

Рисунок 8 – Содержание файла test_figures.py

2. Пример работы программы

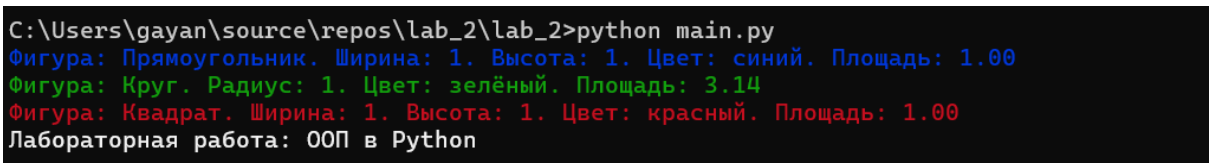


Рисунок 9 – Пример работы программы

Вывод: В ходе выполнения лабораторной работы были успешно освоены ключевые принципы объектно-ориентированного программирования на Python благодаря разработке модульной иерархии геометрических фигур в пакете lab_python_oop, настройке изолированного виртуального окружения и подтверждению корректности логики с помощью модульных тестов unittest.